FATEC PROFESSOR JESSEN VIDAL

GUILHERME BEZERRA JUNQUEIRA

PORTFÓLIO DE APRENDIZAGEM DE PROJETOS INTEGRADO (API)

São José dos Campos
2026

GUILHERME BEZERRA JUNQUEIRA

PORTFÓLIO DE APRENDIZAGEM DE PROJETOS INTEGRADO (API)

Trabalho de Graduação apresentado à Fatec Professor Jessen Vidal, como parte dos requisitos para a obtenção do título de Tecnólogo em Banco de Dados.

Orientador: Prof. Carlos Augusto Lombardi Garcia

São José dos Campos

2026

FOLHA DE APROVAÇÃO

Aprovado em: ____ / ____ / _____

_____

Orientador: Prof. Carlos Augusto Lombardi Garcia

_____

Membro da banca examinadora:

_____

Membro da banca examinadora:

AGRADECIMENTOS

Agradeço ao meu orientador, Prof. Carlos Augusto Lombardi Garcia, pela disponibilidade e pelas orientações na elaboração deste trabalho. Agradeço aos professores da Fatec São José dos Campos, em especial ao Prof. Bertoti, pelos ensinamentos e pelo acompanhamento dos projetos integradores ao longo do curso.

Agradeço aos colegas das equipes de cada semestre, com quem compartilhei as dificuldades e as conquistas de cada projeto, e aos parceiros que apresentaram os desafios reais que deram sentido ao trabalho. Por fim, agradeço à minha família e aos amigos pelo apoio durante toda a graduação.

RESUMO

Este trabalho apresenta o Portfólio de Aprendizagem de Projetos Integrados (API) desenvolvido ao longo do curso de Banco de Dados da Fatec Professor Jessen Vidal, reunindo quatro projetos realizados em equipe do primeiro ao quarto semestre. O objetivo é documentar e analisar a evolução técnica e profissional do autor a partir dos desafios propostos em cada semestre. A metodologia baseou-se na aprendizagem por projetos, com a aplicação do framework Scrum, sprints de três semanas, backlog de produto, critérios de pronto e de aceite, versionamento de código em repositórios do GitHub e entregas incrementais. Foram desenvolvidos um sistema de gestão de turmas, grupos e scores para uma instituição de ensino que aplica a aprendizagem baseada em problemas, em Python; um sistema de leitura e análise de dados climáticos em arquivos CSV, em Java, com interface JavaFX e banco MySQL; uma ferramenta web de captura de notícias e de dados de APIs públicas por web scraping, com back-end em Java; e, como projeto de destaque, um Sistema de Gestão de Manutenções para uma empresa parceira, com Spring Boot, Vue, PostgreSQL e PostGIS. Os resultados evidenciam a progressão da aplicação em Python do primeiro semestre para sistemas web com autenticação, banco de dados geoespacial e práticas estruturadas de trabalho em equipe. Conclui-se que os projetos integradores consolidaram competências em modelagem de dados, desenvolvimento de software e trabalho colaborativo, contribuindo para a preparação do autor para a atuação profissional.

Palavras-chave: Aprendizagem por Projetos; Scrum; Banco de Dados; Desenvolvimento de Software; Portfólio.

ABSTRACT

This work presents the Integrated Projects Learning Portfolio (API) developed during the Database course at Fatec Professor Jessen Vidal, bringing together four team projects carried out from the first to the fourth semester. The goal is to document and analyze the author's technical and professional growth based on the challenges proposed each semester. The methodology relied on project-based learning, applying the Scrum framework, three-week sprints, a product backlog, definitions of ready and done, source code versioning in GitHub repositories and incremental deliveries. The projects were: a system for managing classes, groups and scores for a school that applies problem-based learning, written in Python; a system for reading and analyzing climate data from CSV files, written in Java with a JavaFX interface and a MySQL database; a web tool for collecting news and public API data through web scraping, with a Java back end; and, as the featured project, a Maintenance Management System for a partner company, built with Spring Boot, Vue, PostgreSQL and PostGIS. The results show the progression from the Python application of the first semester to web systems with authentication, a geospatial database and structured teamwork practices. It is concluded that the integrated projects consolidated skills in data modeling, software development and collaboration, helping to prepare the author for professional practice.

Keywords: Project-Based Learning; Scrum; Database; Software Development; Portfolio.

SUMÁRIO

1 INTRODUÇÃO.....7
2 PROJETOS.....8
2.1 Primeiro ano.....8
2.1.1 Primeiro semestre: API-Porygon (2023-2).....8
2.1.2 Segundo semestre: Leitor-de-clima (2024-1).....9
2.2 Segundo ano.....10
2.2.1 Terceiro semestre: FATEC-API-3-SEMESTRE (2024-2).....10
2.2.2 Quarto semestre: CaramelStray-Api-4Semestre (2026-1), API de destaque.....11
3 CONSIDERAÇÕES FINAIS.....14
REFERÊNCIAS.....15
APÊNDICE A – PORTFÓLIO EM FORMATO PDF.....16

1 INTRODUÇÃO

A Aprendizagem por Projetos Integrados (API) é a estratégia adotada pela Fatec nos cursos superiores de tecnologia em que, a cada semestre, equipes de estudantes desenvolvem uma solução de software para um desafio proposto por um parceiro, aplicando de forma conjunta os conhecimentos das disciplinas do período. Este portfólio reúne os projetos integradores realizados pelo autor do primeiro ao quarto semestre do curso de Banco de Dados.

Em todos os projetos, as equipes organizaram o trabalho com o framework Scrum, com papéis de Product Owner, Scrum Master e desenvolvedores, sprints de três semanas, backlog priorizado e código versionado em repositórios do GitHub. Cada semestre apresentou um problema distinto: a gestão de uma metodologia de ensino baseada em problemas, a análise de dados climáticos, a captura de notícias e dados públicos e, por fim, a gestão de manutenções de sistemas distribuídos em escala mundial.

O problema que este trabalho busca responder é como registrar, de forma organizada e comprovável, o aprendizado obtido ao longo dos quatro projetos. Para isso, o portfólio descreve o contexto, o objetivo, a solução, as etapas de desenvolvimento, as tecnologias e a contribuição do autor em cada semestre, destacando a API do quarto semestre, a mais completa do ponto de vista técnico e a mais recente.

A relevância do trabalho está em evidenciar a evolução do autor em modelagem e persistência de dados, em linguagens e frameworks, e em práticas de engenharia de software e de trabalho em equipe, o que contribui para a avaliação da formação e para a apresentação profissional do autor. Os resultados são sintetizados no capítulo de considerações finais.

O documento está organizado da seguinte forma: o capítulo 2 apresenta os projetos em ordem cronológica, com as informações exigidas para cada semestre; o capítulo 3 traz as considerações finais, seguidas das referências e do apêndice com o portfólio em formato PDF.

2 PROJETOS

O Quadro 1 relaciona os quatro projetos integradores, com o período de desenvolvimento e o endereço do respectivo repositório no GitHub. As seções seguintes tratam de cada projeto em ordem cronológica.

Quadro 1 – Projetos integradores do portfólio

Semestre	Projeto	Repositório
1º (2023-2)	API-Porygon	https://github.com/Porygon-Users/API-Porygon
2º (2024-1)	Leitor-de-clima	https://github.com/VCCorpTeam/Leitor-de-clima
3º (2024-2)	FATEC-API-3-SEMESTRE	https://github.com/c137santos/FATEC-API-3-SEMESTRE
4º (2026-1)	CaramelStray-Api-4Semestre (API de destaque)	https://github.com/CaramelStray/CaramelStray-Api-4Semestre

Fonte: elaborado pelo autor (2026).

2.1 Primeiro ano

2.1.1 Primeiro semestre: API-Porygon (2023-2)

Período de desenvolvimento. De 04/09/2023 a 26/11/2023, em quatro sprints de três semanas.

Contexto. A PBLTeX é uma instituição de ensino que aplica a aprendizagem baseada em problemas (PBL, do inglês Problem Based Learning). Seus cursos trabalham com ciclos de entrega, que geram scores parciais dos alunos e, ao final, compõem o FEE (Fator de Ensino Evolutivo).

Objetivo. Desenvolver um sistema de informação que viabilizasse a aplicação dessa metodologia, com controle de turmas, grupos e alunos, carga de alunos, controle dos ciclos de entrega, carga de scores parciais, configuração de parâmetros globais, exportação de dados consolidados e visibilidade objetiva para o acompanhamento dos cursos.

Solução. Um sistema em Python que permite ao administrador cadastrar alunos e professores, criar turmas, definir o tempo total do curso e os ciclos de entrega, criar grupos, fechar turmas, listar alunos e emitir relatórios por turma. Ao professor cabe atribuir scores e feedbacks aos alunos.

Etapas de desenvolvimento. O backlog reuniu nove histórias de usuário, todas concluídas, distribuídas em quatro sprints: cadastros, turmas e ciclos de entrega na Sprint 1; fechamento de turmas e criação de grupos na Sprint 2; atribuição de scores e feedbacks e listagem de alunos na Sprint 3; e menu de funcionalidades e relatórios por turma na Sprint 4. A

equipe Porygon, com oito integrantes, utilizou Scrum com Product Owner e Scrum Master, registrou cada sprint em um arquivo de documentação no repositório e acumulou 205 commits.

Tecnologias. Python, Excel, Git e GitHub, Visual Studio Code, Trello, Canva e Microsoft Teams.

Usabilidade, acessibilidade e organização do conteúdo. A navegação foi organizada em um menu que lista todas as funcionalidades disponíveis ao administrador, o que simplifica o uso do sistema.

Contribuição do autor. Atuei como desenvolvedor da equipe, sem assumir os papéis de Product Owner ou de Scrum Master. Colaborei com contribuições gerais de desenvolvimento do sistema e com a organização do trabalho da equipe, como a divisão das tarefas e o acompanhamento das entregas de cada sprint.

Resultados e aprendizados. O projeto foi a primeira experiência com desenvolvimento em equipe usando Scrum, Git e GitHub. Os principais aprendizados foram a lógica de programação aplicada a regras de negócio reais, a divisão de tarefas em user stories e a comunicação constante com os colegas.

2.1.2 Segundo semestre: Leitor-de-clima (2024-1)

Período de desenvolvimento. De 25/03/2024 a 16/06/2024, em quatro sprints de três semanas.

Contexto. Os dados climáticos de cidades e regiões do estado de São Paulo estavam distribuídos em diversos arquivos CSV, um para cada estação de monitoramento, com formatos variados e valores incorretos por falhas humanas ou de coleta automática.

Objetivo. Criar, em Java e com banco de dados SQL, um sistema que lesse e interpretasse esses arquivos e elaborasse relatórios sobre as médias de temperatura, umidade e quantidade de chuva, além de gráficos do tipo boxplot para uma abordagem mais visual dos dados.

Solução. Um sistema com interface gráfica que carrega e valida os arquivos CSV, identifica a cidade pelo nome do arquivo, armazena cada variável climática separadamente e separa os registros suspeitos (por exemplo, temperaturas acima de 60 °C ou abaixo de -20 °C) para revisão manual, com exclusão ou correção. O sistema gera relatórios de situação por cidade, de valores médios por cidade e período, com periodicidade horária, e de elementos para a plotagem de boxplot.

Etapas de desenvolvimento. O backlog foi composto por treze itens priorizados, entre eles a leitura de arquivos CSV, o diagrama de classes, o gerenciamento de estações, cidades e unidades de medida, a modelagem do banco de dados, a interface de usuário, a unificação dos dados, o tratamento de registros suspeitos, os relatórios e a documentação. Os requisitos foram descritos em histórias de usuário com problema, requisito, descrição e checklist de conclusão. A equipe VCCorp (Vira-lata Caramelo Corp), com oito integrantes, acumulou 244 commits.

Tecnologias. Java, JavaFX, MySQL, Git e GitHub, IntelliJ IDEA e Discord.

Usabilidade, acessibilidade e organização do conteúdo. A interface gráfica em JavaFX permite escolher a cidade, a estação e o período ou a data de análise, e o sistema fornece retorno ao usuário sobre erros encontrados nos arquivos carregados.

Contribuição do autor. Atuei como desenvolvedor da equipe, sem assumir os papéis de Product Owner ou de Scrum Master. Colaborei com contribuições gerais de desenvolvimento do sistema e com a organização do trabalho da equipe, como a divisão das tarefas e o acompanhamento das entregas de cada sprint.

Resultados e aprendizados. O projeto aprofundou o trabalho em equipe e introduziu a integração entre uma aplicação Java, um banco de dados relacional e uma interface gráfica. Os principais aprendizados foram a validação de dados externos, a persistência em MySQL e a revisão do código dos colegas.

2.2 Segundo ano

2.2.1 Terceiro semestre: FATEC-API-3-SEMESTRE (2024-2)

Período de desenvolvimento. De 09/09/2024 a 01/12/2024, em quatro sprints de três semanas, precedidas por uma semana de kick-off (26/08 a 30/08/2024) e encerradas com a Feira de Soluções, em 12/12/2024.

Contexto. O desafio propunha a captura e o armazenamento de notícias estratégicas de portais e de dados relevantes de APIs públicas, como previsões do tempo, de modo a formar um histórico para análises futuras com inteligência artificial.

Objetivo. Construir um sistema web capaz de mapear portais de notícias e APIs de dados, realizar a captura automatizada e contínua das informações e oferecer consultas com filtros avançados, prevendo grande volume de dados.

Solução. Um sistema com cadastro de portais de notícias, APIs, tags e jornalistas, um processo de web scraping que captura e armazena notícias e dados das APIs, relação de tags com regionalismos, indicação de tags relacionadas e telas de consulta de notícias e de APIs com filtros de pesquisa.

Etapas de desenvolvimento. O backlog reuniu quatorze histórias de usuário, todas realizadas: cadastro e captura de dados de APIs e gestão de tags na Sprint 1; regionalismos, portais de notícias e captura de notícias na Sprint 2; edição e consulta de APIs na Sprint 3; e consulta de notícias com filtros e visualização das tags relacionadas na Sprint 4. A equipe Cerberus, com sete integrantes, gerenciou as sprints em um quadro Kanban no GitHub Projects, acompanhou o burndown, adotou o padrão de commits com gitmoji e acumulou 242 commits. O repositório contém ainda o guia de contribuição, o design das telas, o modelo conceitual do banco, a documentação dos campos e o manual do usuário.

Tecnologias. Java no back-end, conforme os requisitos do desafio, e front-end web; Git e GitHub, GitHub Projects, ESLint e Nx, presentes na configuração do repositório.

Usabilidade, acessibilidade e organização do conteúdo. Os requisitos não funcionais determinaram um front-end minimalista, manual do usuário e guia de instalação. As telas foram projetadas previamente em um documento de design, e as consultas contam com filtros de pesquisa para facilitar a localização das informações.

Contribuição do autor. Atuei como desenvolvedor da equipe, sem assumir os papéis de Product Owner ou de Scrum Master. Colaborei com contribuições gerais de desenvolvimento do sistema e com a organização do trabalho da equipe, como a divisão das tarefas e o acompanhamento das entregas de cada sprint.

Resultados e aprendizados. O projeto ampliou a experiência com aplicações web e com a captura automatizada de dados. Os principais aprendizados foram o uso de Kanban para acompanhar as tarefas, a leitura de documentação de APIs públicas e a padronização de commits.

2.2.2 Quarto semestre: CaramelStray-Api-4Semestre (2026-1), API de destaque

Período de desenvolvimento. De 16/03/2026 a 31/05/2026, em três sprints de três semanas, com a Feira de Soluções em 25/06/2026.

Contexto. A empresa parceira, a Altave, opera cerca de cem sistemas distribuídos pelo mundo, cada um exigindo manutenções periódicas conforme suas horas de uso. A duração das intervenções varia com a localização: uma manutenção no Rio de Janeiro leva um dia, enquanto operações em regiões distantes, como a Ásia, demandam mais tempo.

Objetivo. Desenvolver um sistema que apoiasse o planejamento e o registro das manutenções, a organização das viagens dos técnicos e o acompanhamento de prazos contratuais, de modo a priorizar atendimentos e evitar atrasos.

Solução. O Tracker, um Sistema de Gestão de Manutenções que centraliza o cadastro de clientes, sistemas, contratos, ativos e técnicos, e permite planejar e registrar ordens de manutenção com base nas horas de uso dos equipamentos. O sistema organiza viagens e checklists, e oferece dashboard, histórico, calendário, mapa de cobertura e relatórios de manutenções vencidas e próximas do vencimento.

Planejamento e práticas ágeis. O backlog do produto reuniu dezenove histórias de usuário priorizadas e estimadas. As sprints foram planejadas conforme o Quadro 2. A equipe, com oito integrantes, adotou Definition of Ready e Definition of Done por sprint, burn down, vídeos de acompanhamento das sprints 2 e 3, documentação em README e manual do usuário. O fluxo de versionamento seguiu um Git Flow adaptado, com a branch main estável, a branch dev de integração e branches de tarefas (feature e fix) integradas por pull request, e commits no padrão Conventional Commits. O repositório acumulou 465 commits e teve versões marcadas (v1.0 e v1.2).

Quadro 2 – Sprints do projeto do quarto semestre

Sprint	Período	Histórias planejadas
1	16/03 a 05/04/2026	Ordem de manutenção; cadastros de sistemas, contratos, clientes e técnicos.
2	13/04 a 03/05/2026	Execução e criticidade da manutenção; viagens; dashboard de ordens; checklists de preparação e da manutenção; calendário de disponibilidade dos técnicos.
3	11/05 a 31/05/2026	Histórico e relatório de manutenções; cadastros de máquinas e de ativos; rastreamento de técnicos no mapa; ciclo de embarcações e mapa de sistemas instalados.

Fonte: documentação de sprints do repositório (2026).

Arquitetura e tecnologias. O sistema segue arquitetura em camadas, com back-end, front-end e banco de dados executados em contêineres Docker orquestrados por Docker Compose. O back-end é uma API REST em Java 17 com Spring Boot 3.3.5, organizada em controllers, serviços, repositórios, entidades e DTOs, com Spring Data JPA, Spring Security com autenticação por JWT, validação de dados e suporte a Redis. O banco de dados é o PostgreSQL com a extensão geoespacial PostGIS, acessada por meio do Hibernate Spatial para os dados geográficos. O front-end é uma aplicação em Vue 3 com TypeScript e Vite, que utiliza Vue Router, Pinia, FullCalendar, Chart.js, Leaflet, TanStack Table e validação de formulários com VeeValidate e Zod. Os protótipos de interface foram definidos no Figma.

Modelagem de dados. O script do banco de dados, versionado no repositório, define 43 tabelas organizadas em dois grupos: as de cadastro (prefixo tb_cad), como clientes, contratos, funcionários, habilidades, perfis e permissões, catálogos de máquinas, softwares, ativos e tarefas; e as de serviço (prefixo tb_srv), como ordens de serviço, viagens e paradas, checklists, históricos e observações de manutenção e ciclo de embarcações.

Funcionalidades. Conforme o manual do usuário, o sistema é dividido em módulos: clientes e contatos, certificações e habilidades, técnicos, máquinas, sistemas, contratos, ordens de serviço (cadastro, seleção de máquina e de técnico, visualização detalhada com linha do tempo e checklists), preparação de viagem, histórico de manutenções, ativos, calendário, mapa de cobertura e dashboard. As histórias do ciclo de embarcações e do mapa de sistemas instalados, que a documentação de sprints ainda registra como planejadas, foram entregues e estão presentes no sistema, com telas de mapa e de embarcações atracadas.

Usabilidade, acessibilidade e organização do conteúdo. O sistema atende a perfis distintos: o gestor acessa os cadastros, o dashboard, o calendário e os relatórios, e o técnico acessa a tela Minhas Ordens, com as ordens atribuídas, os checklists e a preparação de viagem. O calendário oferece visões mensal e semanal e filtros por criticidade, status da ordem e técnico. O cadastro de preparação de viagem foi dividido em cinco etapas (identificação, deslocamento, estimativas, paradas e revisão), reduzindo a carga de informação em cada tela. O conteúdo é

organizado em módulos com busca, indicadores e ações por tela, e documentado em um manual do usuário com índice.

Contribuição do autor. O histórico do repositório registra 13 commits do autor, entre 26/04/2026 e 29/05/2026, concentrados no front-end em Vue. Foram desenvolvidos a tela de relatório de manutenção; a listagem de ordens do técnico; os cards do dashboard, inclusive o estado sem dados; o modal de detalhe de evento no calendário de técnicos; a listagem de embarcações atracadas com destaque; o cadastro de preparação de viagem em formato de modal; e a atualização periódica (polling) com indicação visual de status no mapa. O autor também realizou ajustes nas rotas e na barra lateral de navegação, pequenas alterações no back-end (DTOs, repositório de ativos e serviço de técnicos) e atualizações no script do banco de dados.

Resultados e aprendizados. Como desenvolvedor, o autor trabalhou em uma aplicação de maior porte, com back-end, banco de dados geoespacial, front-end em Vue e execução em contêineres. Os principais aprendizados foram o fluxo de branches e pull requests, o consumo de uma API REST autenticada e o desenvolvimento de interfaces com calendário, gráficos e mapas. Na Feira de Soluções, a equipe apresentou o produto aos clientes e praticou formas de explicar o sistema e o seu valor para quem não é da área técnica.

3 CONSIDERAÇÕES FINAIS

Os quatro projetos integradores mostram uma progressão clara de complexidade. No primeiro semestre, o foco foi a lógica de programação e a modelagem de regras de negócio de um sistema acadêmico em Python. No segundo, somaram-se a persistência em banco relacional MySQL, a leitura e a validação de dados externos e uma interface gráfica em Java. No terceiro, o trabalho passou a envolver uma aplicação web com captura automatizada de dados de portais e APIs públicas. No quarto, o projeto reuniu uma API REST com autenticação, um banco de dados com recursos geoespaciais, um front-end rico em calendário, gráficos e mapas, e a execução em contêineres.

Também evoluíram as práticas de engenharia de software. Os critérios de pronto e de aceite, o acompanhamento por burndown, o fluxo de branches com pull requests, os padrões de commit e a documentação em manual do usuário tornaram-se mais estruturados a cada semestre, e a convivência em equipes de sete a oito integrantes, com papéis definidos, exigiu comunicação e integração constantes do código.

Entre as contribuições do autor, destacam-se, no quarto semestre, as telas de relatório, dashboard, calendário, preparação de viagem e mapa do sistema Tracker, que aplicaram na prática os conhecimentos de modelagem de dados e de desenvolvimento de interfaces adquiridos no curso. Nos três primeiros semestres, atuei como desenvolvedor das equipes, colaborando com o código e com a organização das tarefas, sem ocupar as funções de Product Owner ou de Scrum Master.

Entre os desafios, estiveram a adaptação a tecnologias novas a cada semestre, a integração do código de vários desenvolvedores e a conciliação dos estudos com o trabalho. Eles foram superados com a divisão clara das tarefas, a revisão do código por pull requests, a comunicação frequente com a equipe e o apoio dos professores.

Os projetos integradores aproximaram o autor da rotina de uma equipe de desenvolvimento: metodologia ágil, controle de versão, documentação e entrega para um cliente real. Essa experiência contribuiu para a formação profissional na área de banco de dados e desenvolvimento de software.

REFERÊNCIAS

CARAMELSTRAY. CaramelStray-Api-4Semestre: sistema de gestão de manutenções (Tracker). GitHub, 2026. Disponível em: https://github.com/CaramelStray/CaramelStray-Api-4Semestre. Acesso em: 19 out. 2026.

C137SANTOS. FATEC-API-3-SEMESTRE: captura de notícias e dados de APIs públicas. GitHub, 2024. Disponível em: https://github.com/c137santos/FATEC-API-3-SEMESTRE. Acesso em: 19 out. 2026.

GITMOJI. Gitmoji: an emoji guide for your commit messages. Disponível em: https://gitmoji.dev/. Acesso em: 19 out. 2026.

OPENJFX. JavaFX. Disponível em: https://openjfx.io/. Acesso em: 19 out. 2026.

ORACLE. MySQL reference manual. Disponível em: https://dev.mysql.com/doc/. Acesso em: 19 out. 2026.

PORYGON-USERS. API-Porygon: API do 1º semestre de Banco de Dados. GitHub, 2023. Disponível em: https://github.com/Porygon-Users/API-Porygon. Acesso em: 19 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL documentation. Disponível em: https://www.postgresql.org/docs/. Acesso em: 19 out. 2026.

PYTHON SOFTWARE FOUNDATION. Python documentation. Disponível em: https://docs.python.org/3/. Acesso em: 19 out. 2026.

SCHWABER, Ken; SUTHERLAND, Jeff. O guia do Scrum. 2020. Disponível em: https://scrumguides.org/. Acesso em: 19 out. 2026.

SPRING. Spring Boot. Disponível em: https://spring.io/projects/spring-boot. Acesso em: 19 out. 2026.

VCCORPTEAM. Leitor-de-clima: leitura e análise de dados climáticos. GitHub, 2024. Disponível em: https://github.com/VCCorpTeam/Leitor-de-clima. Acesso em: 19 out. 2026.

VUE.JS. Vue.js: the progressive JavaScript framework. Disponível em: https://vuejs.org/. Acesso em: 19 out. 2026.

APÊNDICE A – PORTFÓLIO EM FORMATO PDF

O portfólio completo, em formato PDF, acompanha este trabalho como arquivo anexo.